

Motor de Trazabilidad del Café sobre una Cadena de Bloques

Implementado con Estructuras de Datos Propias en C++

Karen Sofía Cruz Alvarado y Nicolas Garzón Ballen
Ingeniería de Sistemas y Computación — Programación III
Universidad Pedagógica y Tecnológica de Colombia
Facultad Seccional Sogamoso, Colombia
karen.cruz11@uptc.edu.co, nicolas.garzon02@uptc.edu.co

Resumen—Este documento presenta el diseño y la implementación de un motor de trazabilidad inmutable para la cadena de suministro del café de una cooperativa de caficultores de Boyacá, construido como una cadena de bloques (*blockchain*) en C++ sin contenedores de la STL. La cadena se modela como una lista doblemente enlazada genérica de bloques, y cada bloque contiene una lista enlazada sencilla genérica de transacciones (eventos de custodia). El enlace entre bloques es el hash del bloque anterior, calculado con FNV-1a de 64 bits, y la escritura de un bloque exige una prueba de trabajo con dificultad configurable. El sistema ofrece un menú de consola con siete opciones (registro, minado, navegación, validación de integridad, trazabilidad de lotes, simulación de manipulación y salida), gestión de memoria en dos niveles verificada con Valgrind y, como bonos, persistencia en archivo e interfaz gráfica con Raylib. Se demuestra que la alteración de un dato histórico es detectada e identificada por el índice exacto del bloque comprometido.

Palabras clave—*blockchain*, trazabilidad, listas enlazadas, hash, prueba de trabajo, FNV-1a, C++, café.

I. INTRODUCCIÓN

Los compradores internacionales de café de origen pagan un sobreprecio por lotes trazables: desean verificar que el grano proviene de fincas certificadas y que pasó por cada etapa (cosecha, beneficio, trilla, almacenamiento, transporte y exportación) sin adulteraciones. Hoy la cooperativa lleva ese registro en hojas de cálculo, un medio que no ofrece garantía de integridad: cualquier intermediario podría modificar un peso, cambiar un origen o borrar un evento de custodia sin dejar rastro.

Se requiere, por tanto, un registro *append-only* (solo se agrega, nunca se modifica) y a prueba de manipulación. Esa es la garantía que ofrece una cadena de bloques. En este trabajo se construye el motor local de un nodo: la cadena, los bloques, las transacciones, el *hashing*, la prueba de trabajo y la validación de integridad. La parte distribuida (replicación, consenso y tolerancia a fallos) corresponde a Sistemas Distribuidos y queda fuera del alcance.

I-A. Objetivos

- Implementar una cadena de bloques con una lista doblemente enlazada propia de bloques y, dentro de cada bloque, una lista enlazada sencilla propia y genérica de transacciones.
- Calcular el hash de cada bloque sobre todos sus campos (incluidas sus transacciones) y encadenar los bloques por hash.
- Implementar prueba de trabajo con dificultad configurable.
- Validar la integridad de toda la cadena y detectar manipulaciones indicando el primer bloque comprometido.
- Reconstruir la historia de vida de un lote recorriendo los dos ejes de la estructura anidada.
- Gestionar la memoria en dos niveles sin fugas.

II. MARCO TEÓRICO

II-A. Cadena de bloques

Una cadena de bloques es un registro (*ledger*) ordenado cronológicamente, formado por bloques encadenados en el que cada bloque guarda una huella criptográfica del anterior [1]. Estructuralmente es una lista enlazada, pero el enlace no es un puntero cualquiera sino el hash del bloque previo.

II-B. Puntero frente a hash

Un puntero indica dónde está el vecino, pero no certifica qué contiene: si el contenido cambia, el puntero sigue igual. Un hash, en cambio, certifica el contenido: si el bloque anterior cambia en un solo carácter, su hash cambia y el enlace deja de coincidir. Una *blockchain* es, en esencia, una lista enlazada con memoria de lo que cada eslabón contenía.

II-C. Función hash

Una función hash criptográfica $H(x)$ recibe una entrada de cualquier tamaño y produce una salida de tamaño fijo. Las propiedades que sostienen la inmutabilidad son: determinismo, efecto avalancha, resistencia a preimagen y resistencia a

colisiones. En este proyecto el hash de un bloque se define como

$$h = H(\text{indice} \parallel \text{timestamp} \parallel \text{datos} \parallel \text{hashPrevio} \parallel \text{nonce}). \quad (1)$$

II-D. Prueba de trabajo

Para encarecer la manipulación se exige que el hash de un bloque comience con d ceros (la dificultad). Como la salida de H no puede elegirse, se prueban valores del *nonce* por fuerza bruta hasta cumplir la condición; verificarla, en cambio, cuesta un solo cálculo [3]. Si el hash se expresa en hexadecimal, cada dígito aporta 4 bits y la probabilidad de que un intento cumpla la condición es 16^{-d} , por lo que el número esperado de intentos es

$$E[\text{intentos}] = 16^d. \quad (2)$$

II-E. Inmutabilidad

Si un atacante altera una transacción del bloque k , el hash recalculado de ese bloque deja de coincidir con el almacenado y, además, el bloque $k+1$ conserva un *hashPrevio* que ya no corresponde al hash real del bloque k . Para ocultar el fraude tendría que volver a minar el bloque k y todos los siguientes, un costo creciente con la longitud de la cadena.

III. DISEÑO DE LA SOLUCIÓN

III-A. Modelo de datos

La solución se organiza en dos estructuras anidadas (Fig. 1): una lista doblemente enlazada de bloques y, en cada bloque, una lista enlazada sencilla de transacciones.

- **Transacción:** evento de custodia con identificador del lote, etapa, actor responsable, peso en kg y fecha.
- **Bloque:** índice, *timestamp*, *hashPrevio*, *nonce*, hash propio y su lista de transacciones.
- **Cadena:** lista doble de bloques, dificultad global y un “bloque abierto” (aún no minado) que acumula transacciones.

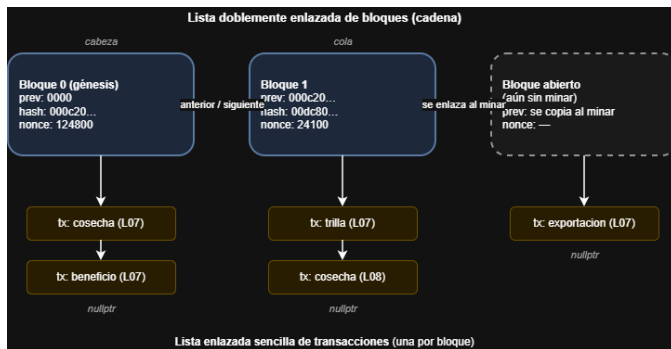


Figura 1. Estructura anidada: lista doble de bloques y lista sencilla de transacciones por bloque.

III-B. Diagrama de clases UML

El diseño de referencia del enunciado se adaptó con una decisión principal: en lugar de reimplementar los enlaces en cada clase, se reutilizan dos plantillas genéricas propias. La cadena es una `DoubleLinkedList<Bloque*>` y la lista de

Cuadro I
CLASES DEL SISTEMA Y SU RESPONSABILIDAD

Clase	Responsabilidad
Transaccion	Dato puro de un evento de custodia; <code>aTexto()</code> entrega la cadena que entra al hash.
NodoSimple<T>	Nodo de la lista sencilla: dato y puntero al siguiente.
ListaSimple<T>	Lista enlazada sencilla genérica con inserción al final en $O(1)$ (mantiene cabeza y cola). Su destructor libera los nodos.
Node<T>	Nodo de la lista doble: <code>info</code> , <code>next</code> y <code>previous</code> .
DoubleLinkedList<T>	Lista doblemente enlazada genérica (inserción, búsqueda, eliminación, recorrido).
Bloque	Metadatos, lista de transacciones, cálculo de hash y minado.
Blockchain	Administra la cadena, el bloque abierto, la validación, la trazabilidad, la simulación de manipulación y la persistencia.
InterfazGrafica(Bono)	Interfaz Raylib que dibuja y navega la cadena.

transacciones es una `ListaSimple<Transaccion>`. Por ello `Bloque` no guarda punteros anterior/siguiente: los provee el `Node<Bloque*>` de la lista doble. La Fig. 2 muestra el diagrama resultante y la Tabla I resume cada clase.

III-C. Organización del código

Cada clase se separa en archivos `.h` (declaración) y `.cpp` (implementación). El núcleo (`Transaccion`, `Bloque`, `Blockchain`) se compila como una biblioteca estática compartida por dos ejecutables: `blockchain_consola` (menú obligatorio) y `blockchain_gui` (bono). La construcción se automatiza con CMake. Como `DoubleLinkedList<T>` es una plantilla con la implementación en un `.cpp` separado, `Blockchain.cpp` la incluye e instancia explícitamente para `Bloque*`.

IV. IMPLEMENTACIÓN

IV-A. Función hash (FNV-1a de 64 bits)

El hash se calcula sobre la representación en texto del bloque, que concatena en orden: índice, *timestamp*, el texto de cada transacción de su lista sencilla, el hash previo y el *nonce* (ecuación 1). Con ello cualquier cambio en una transacción altera el hash del bloque. La salida son 16 dígitos hexadecimales.

Listing 1. Cálculo del hash con FNV-1a (`Bloque.cpp`).

```
std::string Bloque::calcularHash() const {
    std::string texto = aTexto();
    unsigned long long h = 14695981039346656037ULL; // offset
    const unsigned long long primo = 1099511628211ULL;
    for (unsigned char c : texto) {
        h ^= c;
        h *= primo;
    }
    std::ostringstream oss;
    oss << std::hex << std::setfill('0') << std::setw(16)
    << h;
```

Diagrama de clases — Blockchain de trazabilidad del café (C++)

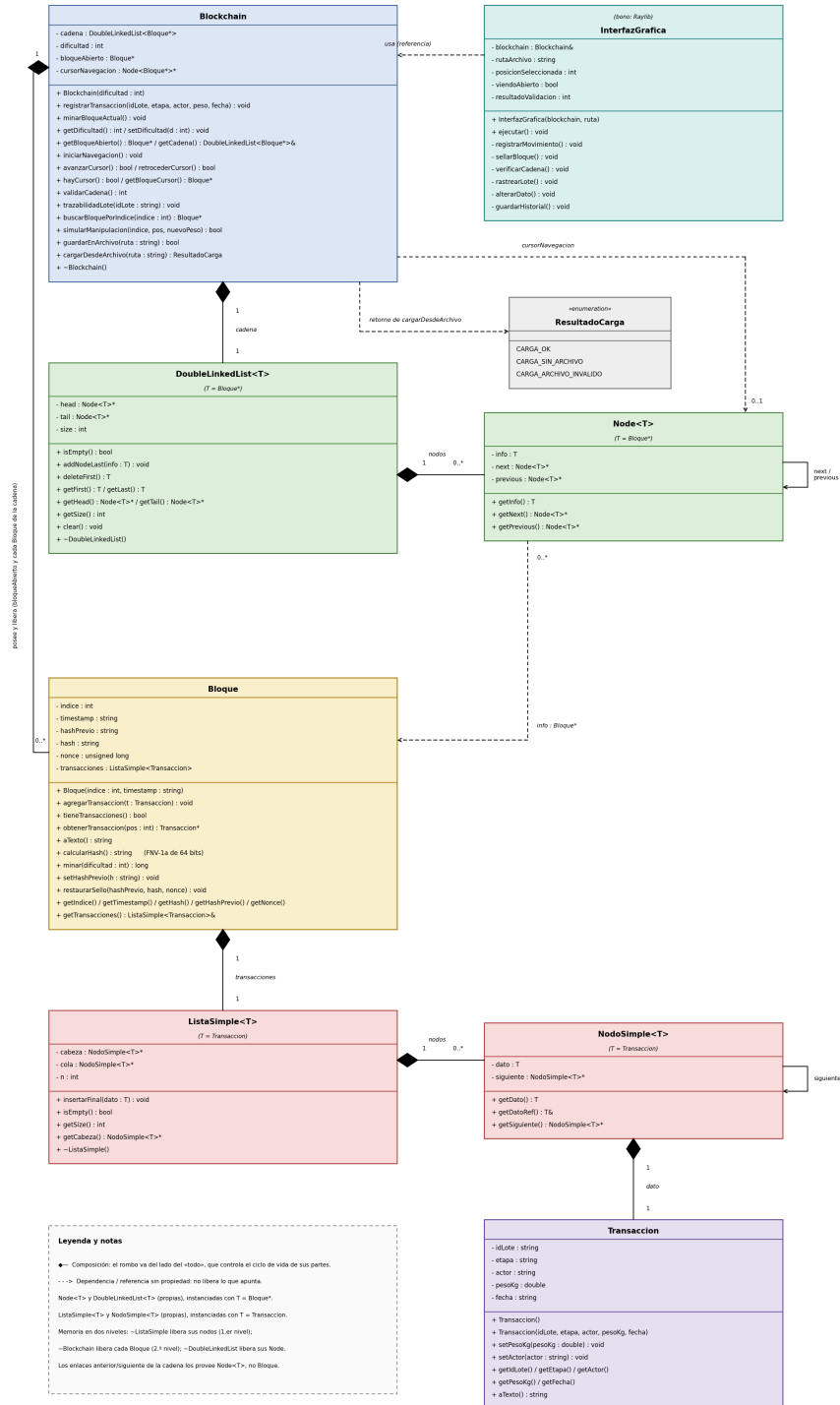


Figura 2. Diagrama de clases UML del sistema.

Cuadro II
COSTO DEL MINADO SEGÚN LA DIFICULTAD

d	Intentos esperados (16^d)	Intentos medidos	Tiempo (s)
1	16	[completar]	[completar]
2	256	[completar]	[completar]
3	4 096	[completar]	[completar]
4	65 536	[completar]	[completar]
5	1 048 576	[completar]	[completar]
6	16 777 216	[completar]	[completar]

```
return oss.str();
}
```

Dos detalles de implementación resultan decisivos. Primero, el relleno con ceros a la izquierda hasta 16 dígitos: sin él, `std::hex` no imprime ceros iniciales y la condición “el hash empieza con d ceros” nunca se cumpliría, dejando el minado en un bucle infinito. Segundo, el peso se serializa con `setprecision(17)` para que un cambio pequeño (por ejemplo, de 18000.75 a 18000.76) modifique el texto y, por tanto, el hash.

IV-A0a. Limitaciones frente a SHA-256: FNV-1a es un hash *didáctico*: es determinista y presenta efecto avalancha, pero no es criptográfico. Con solo 64 bits, una colisión puede hallarse con unos 2^{32} intentos (paradoja del cumpleaños), frente a 2^{128} de SHA-256 con 256 bits; además no fue diseñado para resistir ataques de preimagen, de modo que un atacante podría fabricar contenido que produzca un hash dado. En un sistema real se usaría SHA-256, como en Bitcoin [1].

IV-B. Prueba de trabajo

`Bloque::minar(d)` parte de `nonce = 0` y lo incrementa hasta que `calcularHash()` comience con d ceros, devolviendo la cantidad de intentos. La dificultad es una regla de toda la cadena, por lo que vive en Blockchain y es configurable entre 1 y 6.

Listing 2. Minado por fuerza bruta (Bloque.cpp).

```
long Bloque::minar(int dificultad) {
    std::string objetivo(dificultad, '0');
    long intentos = 0;
    nonce = 0;
    hash = calcularHash();
    intentos++;
    while (hash.compare(0, dificultad, objetivo) != 0) {
        nonce++;
        hash = calcularHash();
        intentos++;
    }
    return intentos;
}
```

El costo esperado crece exponencialmente con d (ecuación 2); la Tabla II resume el costo teórico y deja espacio para registrar los intentos medidos.

IV-C. Funcionalidades del menú

El menú de consola implementa las siete opciones del enunciado. La lectura de datos valida cada entrada (números completos y en rango, fechas en formato AAAA-MM-DD,

campos obligatorios) y ofrece las etapas habituales como lista numerada.

IV-C1. Opción 1: registrar una transacción: Pide lote, etapa, actor, peso y fecha, crea una `Transaccion` y la inserta al final de la lista sencilla del bloque abierto. Un bloque acumula varias transacciones antes de cerrarse.

```
=====
MENU PRINCIPAL
=====
Bloques sellados: 0 | Movimientos pendientes de sellar: 1
-----
1. Registrar un movimiento del cafe
2. Sellar los movimientos pendientes (minar bloque)
3. Ver el historial de bloques (navegar la cadena)
4. Verificar que ningun registro fue alterado (validar)
5. Consultar el recorrido de un lote de cafe (trazabilidad)
6. Prueba de seguridad: alterar un dato (simular manipulacion)
7. Salir (el historial se guarda automaticamente)
-----
Paso a paso: registre movimientos (1), luego selle (2) y por ultimo
consulte o verifique (3, 4 o 5).

Elija una opcion (1-7):
Complete los datos del movimiento.

Codigo del lote de cafe (ej. L07):L07

Etapas del proceso:
1. Cosecha
2. Beneficio
3. Trilla
4. Almacenamiento
5. Transporte
6. Exportacion
7. Otra (escribir el nombre)
Elija la etapa (1-7):4

Responsable de esta etapa (finca, cooperativa, transportador...)
Ej. Finca La Esperanza:Finca La Esperanza

Peso registrado en kg (decimales con punto, ej. 120.5):120.5

Fecha del movimiento (AAAA-MM-DD) [Enter = hoy, 2026-10-06]:2026-03-10

[OK] Movimiento registrado: lote L07 | cosecha | Finca La Esperanza | 120.50 kg | 2026-03-10
Queda pendiente de sellar en el bloque #0. Cuando termina de registrar,
```

Figura 3. Menú principal y registro de una transacción (opción 1).

IV-C2. Opción 2: minar el bloque actual: Rechaza el minado de un bloque vacío (lanza una excepción). Copia en el bloque abierto el `hashPrevio` (el hash del último bloque, o "0000" si es el génesis), ejecuta la prueba de trabajo, enlaza el bloque con `addNodeLast`, informa los intentos y abre un bloque nuevo y vacío con el índice siguiente.

```
Elija una opcion (1-7):2

=====
SELLAR LOS MOVIMIENTOS PENDIENTES
=====
Se sellaran 2 movimiento(s) en el bloque #0.
Una vez sellado, ya no se podra modificar sin que el sistema lo detecte.
Sellando, por favor espere...

Detalle tecnico -> Bloque 0 minado en 124801 intento(s). Hash: 000c206803273ea7

[OK] Bloque #0 sellado y guardado en el historial.
```

Figura 4. Minado de un bloque (opción 2).

IV-C3. Opción 3: navegar la cadena: Un cursor de tipo `Node<Bloque*>` permite avanzar con `getNext()` o retroceder con `getPrevious()`. En cada posición se muestran índice, *timestamp*, hash, hash previo y se recorre la lista sencilla para listar las transacciones. Los extremos se manejan devolviendo `false` cuando no existe anterior (génesis) o siguiente (último bloque).

```
=====
HISTORIAL DE BLOQUES
=====

-----
BLOQUE #0  (bloque 1 de 2)  - bloque inicial
-----

Sellado el:      Tue Oct 06 18:36:25 2026
Huella digital (hash): 000c206803273ea7
Huella del bloque anterior: 0000 (no tiene anterior)
Clave de sellado (nonce): 124800

Movimientos registrados en este bloque:
1. 2026-03-10 | Lote L07 | cosecha | Finca La Esperanza | 120.50 kg
2. 2026-03-12 | Lote L07 | beneficio | Cooperativa | 118.00 kg

[S] Siguiente bloque  [A] Bloque anterior  [Q] Volver al menu:A

[!] Este es el primer bloque, no hay uno anterior.

-----
BLOQUE #1  (bloque 2 de 2)
-----

Sellado el:      Tue Oct 06 18:42:18 2026
Huella digital (hash): 00dc800a35be0192
Huella del bloque anterior: 000c206803273ea7
Clave de sellado (nonce): 24100

Movimientos registrados en este bloque:
1. 2026-03-20 | Lote L07 | trilla | Cooperativa | 110.00 kg
2. 2026-03-11 | Lote L08 | cosecha | Finca El Roble | 95.00 kg

[S] Siguiente bloque  [A] Bloque anterior  [Q] Volver al menu:S

[!] Este es el ultimo bloque, no hay uno siguiente.
```

Figura 5. Navegación con cursor sobre la lista doble (opción 3).

IV-C4. Opción 4: validar la integridad: Recorre la cadena desde el génesis. Para cada bloque verifica (a) que el hash recalculado coincide con el almacenado y (b) que su `hashPrevio` es igual al hash real del bloque anterior. Retorna `-1` si la cadena es válida o el índice del primer bloque comprometido, deteniéndose allí.

Listing 3. Validación de integridad (Blockchain.cpp).

```
int Blockchain::validarCadena() {
    if (cadena.isEmpty()) return -1;
    Node<Bloque*> actual = cadena.getHeader();
    std::string hashRealAnterior = "0000";
    while (actual != nullptr) {
        Bloque* b = actual->getInfo();
        std::string hashReal = b->calcularHash();
        if (hashReal != b->getHash()) return b->getIndice();
        if (b->getHashPrevio() != hashRealAnterior)
            return b->getIndice();
        hashRealAnterior = hashReal;
        actual = actual->getNext();
    }
    return -1;
}
```

```
hashRealAnterior = hashReal;
actual = actual->getNext();
}
return -1;
}
```

```
Elija una opcion (1-7):4

=====
VERIFICAR LA INTEGRIDAD DE LOS REGISTROS
=====
Revisando 2 bloque(s), desde el primero hasta el ultimo...

[OK] Todo en orden: ningun registro fue alterado.
```

Figura 6. Validación de una cadena íntegra (opción 4).

IV-C5. Opción 5: trazabilidad de un lote: Dado un identificador de lote, recorre el primer eje (bloque por bloque en la lista doble) y, dentro de cada bloque, el segundo eje (la lista sencilla), filtrando las transacciones del lote. Imprime en orden cronológico la historia de vida del lote indicando el bloque donde quedó registrado cada evento, e incluye también las transacciones pendientes del bloque abierto (marcadas como “aún no minado”).

```
Elija una opcion (1-7):5

=====
RECORRIDO DE UN LOTE DE CAFE
=====
Codigo del lote a consultar (ej. L07):L07

Trazabilidad del lote L07:
[Bloque 0] cosecha - actor: Finca La Esperanza, peso: 120.50 kg, fecha: 2026-03-10
[Bloque 0] beneficio - actor: Cooperativa, peso: 118.00 kg, fecha: 2026-03-12
[Bloque 1] trilla - actor: Cooperativa, peso: 110.00 kg, fecha: 2026-03-20
[Bloque 2, aun no minado] exportacion - actor: Exportadora, peso: 108.00 kg, fecha: 2026-04-05
```

Figura 7. Historia de vida de un lote (opción 5).

IV-C6. Opción 6: simular una manipulación: El usuario elige un bloque histórico y una de sus transacciones y cambia su peso *sin volver a minar*. Al ejecutar nuevamente la validación, el hash recalculado del bloque ya no coincide con el almacenado y el sistema reporta el índice exacto donde se rompió la cadena (Fig. 8).

IV-C7. Opción 7: salir: Libera toda la memoria. Si la entrada se cierra de forma inesperada (`Ctrl+D`), se lanza una excepción propia que se propaga hasta `main`, de modo que los destructores liberan igualmente todo.

IV-D. Gestión de memoria en dos niveles

- Primer nivel (bloque).** Bloque guarda su `ListaSimple<Transaccion>` por valor; al destruirse un bloque se invoca automáticamente el destructor de la lista, que recorre y libera todos sus nodos. La lista tiene el constructor de copia y la asignación deshabilitados para evitar dobles `delete`.
- Segundo nivel (cadena).** `DoubleLinkedList<Bloque*>` libera sus


```
Elija una opcion (1-7):7

[i] Tiene 1 movimiento(s) sin sellar. Se guardaran como
    pendientes y podra sellarlos la proxima vez que abra el sistema.

[OK] Historial guardado en cadena_cafe.txt (2 bloque(s) sellado(s), 1 pendiente(s)).

Gracias por usar el sistema de trazabilidad. Hasta pronto.

Process finished with exit code 0
```

```
=====
VERIFICAR LA INTEGRIDAD DE LOS REGISTROS
=====
Revisando 2 bloque(s), desde el primero hasta el ultimo...

[ALERTA] Se detecto una alteracion en el bloque #0.
    Los datos de ese bloque (y de los siguientes) ya no son confiables.
```

Figura 8. Demostración de detección de manipulación: validación antes (izquierda) y después (derecha) de alterar un bloque.

nodos pero no los Bloque apuntados; por eso ~Blockchain() elimina el bloque abierto y extrae cada bloque con deleteFirst() para hacerle delete, activando así el primer nivel.

Listing 4. Destructor de la cadena (Blockchain.cpp).

```
Blockchain::~Blockchain() {
    delete bloqueAbierto;
    while (!cadena.isEmpty()) {
        Bloque* b = cadena.deleteFirst();
        delete b;
    }
}
```

IV-E. Bonos

IV-E1. Persistencia en archivo (+5 %): La cadena completa se guarda en cadena_cafe.txt, un dato por línea, con encabezado CADENA_CAFE v1, dificultad, bloques sellados (con su hash y nonce originales), transacciones pendientes del bloque abierto y marca FIN. Al cargar **no se vuelve a minar**: se restaura cada sello tal cual y luego se puede validar, de modo que si alguien edita el archivo a mano la alteración es detectada. La carga es robusta ante archivos incompletos o dañados (devuelve CARGA_ARCHIVO_INVALIDO y deja la cadena vacía y sin fugas).

CADENA_CAFE v1	BLOQUE
2	1
2	Tue Oct 06 18:42:18 2026
BLOQUE	000c206803273ea7
0	00dc800a35be0192
Tue Oct 06 18:36:25 2026	24100
0000	2
000c206803273ea7	L07
124800	trilla
2	Cooperativa
L07	110
cosecha	2026-03-20
Finca La Esperanza	L08
120.5	cosecha
2026-03-10	Finca El Roble
L07	95
beneficio	2026-03-11
Cooperativa	PENDIENTES
118	1
2026-03-12	L07
	exportacion
	Exportadora
	108
	2026-04-05
	FIN

Figura 9. Formato del archivo de persistencia.

IV-E2. Interfaz gráfica con Raylib (+10 %):

InterfazGrafica dibuja los bloques enlazados con doble flecha y sus transacciones, y permite navegar la cadena, registrar movimientos, sellar bloques, verificar la cadena, rastrear un lote, alterar un dato para probar la detección y guardar el historial. No reimplementa lógica: toda acción pasa por Blockchain, y la navegación usa el mismo cursor de la opción 3 de la consola. Ambos programas comparten el archivo de historial.

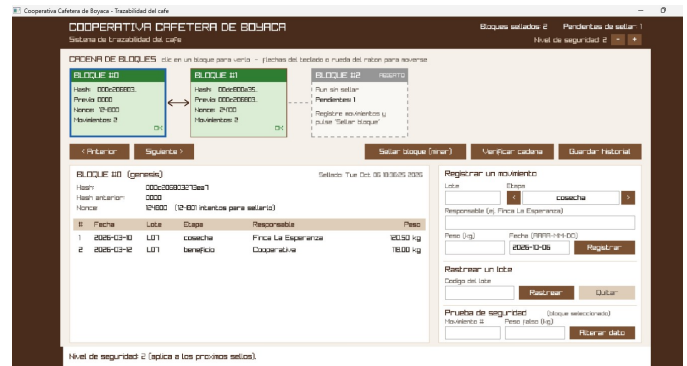


Figura 10. Interfaz gráfica con Raylib: cadena de bloques y navegación.

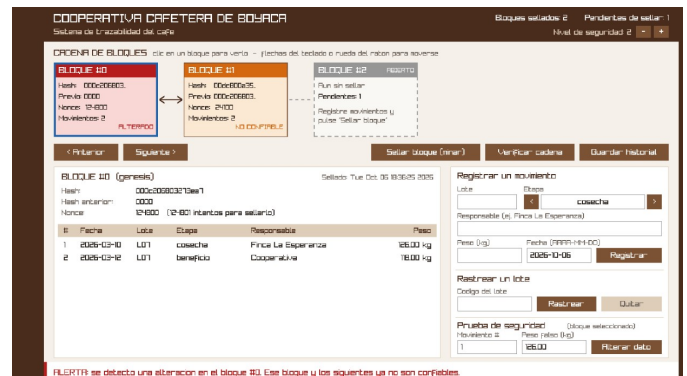


Figura 11. Interfaz gráfica: detección de un bloque alterado / rastreo de lote.

V. PRUEBAS Y RESULTADOS

V-A. Escenario de prueba

Se usó el ejemplo del enunciado con dificultad 2: el lote L07 se registra en cosecha (Finca La Esperanza, 120.5 kg) y beneficio (Cooperativa, 118 kg); se mina el bloque 0; se registran trilla del lote L07 (110 kg) y cosecha del lote L08

(Finca El Roble, 95 kg); se mina el bloque 1; se registra la exportación del lote L07 (108 kg). Luego se navega la cadena, se consulta la trazabilidad de L07, se valida, se altera el peso de la primera transacción del bloque 1 a 900 kg y se vuelve a validar. El archivo `datos_prueba.txt` automatiza este flujo por entrada estándar.

V-B. Detección de manipulación

Antes de la alteración la validación devuelve -1 (cadena íntegra). Tras modificar el peso sin volver a minar, el hash recalculado del bloque alterado difiere del almacenado y la validación devuelve el índice de ese bloque, deteniéndose allí. Como los bloques posteriores guardan el hash original, también quedarían inconsistentes: basta con reportar el primero para localizar el origen del fraude (Fig. 8).

V-C. Verificación de memoria con Valgrind

La prueba se ejecuta sobre la versión de consola:

```
valgrind --leak-check=full \
./build/blockchain_consola < datos_prueba.txt
```

La prueba no se hace sobre la interfaz gráfica porque Valgrind reporta allí memoria de los controladores de video (OpenGL/X11) ajena al proyecto.

```
==7722== Memcheck, a memory error detector
==7722== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==7722== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==7722== Command: ./build/blockchain_consola < datos_prueba.txt
==7722==
==7722== HEAP SUMMARY:
==7722==   in use at exit: 232,013 bytes in 3,510 blocks
==7722==   total heap usage: 75,155 allocs, 71,645 frees, 114,757,759 bytes allocated
==7722==
==7722== 112 (56 direct, 56 indirect) bytes in 1 blocks are definitely lost in loss record 59 of 98
==7722==   at 0x4A8CE43: calloc (vg_replace_malloc.c:1678)
==7722==   by 0x6AB0BA0: ???
==7722==   by 0x653ADF8: ???
==7722==   by 0x6520875: ???
==7722==   by 0x6532096: ???
==7722==   by 0x64CF55E: ???
==7722==   by 0x64D2281: ???
==7722==   by 0x64ACEA4: ???
==7722==   by 0x646FF67: ???
==7722==   by 0x64B5ACD: ???
==7722==   by 0x64B21E7: ???
==7722==   by 0x643D14B: ???
==7722==
==7722== LEAK SUMMARY:
==7722==   definitely lost: 56 bytes in 1 blocks
==7722==   indirectly lost: 56 bytes in 1 blocks
==7722==   possibly lost: 0 bytes in 0 blocks
==7722==   still reachable: 231,853 bytes in 3,507 blocks
==7722==   suppressed: 48 bytes in 1 blocks
==7722==
==7722== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Figura 12. Resultado de Valgrind: sin fugas de memoria.

V-D. Compilación

El proyecto se compila con CMake y estándar C++17:

```
cmake -S . -B build
cmake --build build
./build/blockchain_consola
./build/blockchain_gui
```

Para compilar solo la consola se usa `-DCON_INTERFAZ_GRAFICA=OFF`. Si Raylib no está instalado, CMake la descarga la primera vez.

V-E. Análisis de complejidad

Sea B el número de bloques y T el número total de transacciones.

- Insertar una transacción: $O(1)$ (la lista sencilla mantiene puntero a la cola).
- Enlazar un bloque minado: $O(1)$ (`addNodeLast`).

- Minar: $O(16^d \cdot m)$ en promedio, con m el tamaño del texto del bloque.
- Validar la cadena y consultar la trazabilidad de un lote: $O(B + T)$, pues se visitan todos los bloques y todas las transacciones.
- Navegar un paso del cursor: $O(1)$.

VI. CONCLUSIONES

- La estructura anidada (lista doble de bloques y lista sencilla de transacciones) modela fielmente una block-chain y permite recorrer la historia en ambos sentidos y reconstruir la vida de un lote recorriendo los dos ejes.
- Sustituir el puntero por el hash como enlace convierte una lista enlazada común en un registro a prueba de manipulación: al alterar un dato, la validación señala el índice exacto del bloque comprometido.
- Hacer genéricas las listas con plantillas permitió reutilizarlas sin acoplarlas a Bloque ni a Transaccion.
- La prueba de trabajo hace que el costo del minado crezca de forma exponencial con la dificultad, encareciendo la reescritura de la historia.
- La gestión de memoria en dos niveles, verificada con Valgrind, evita fugas porque cada propietario libera lo que contiene.
- FNV-1a es adecuado con fines didácticos, pero un sistema real debe usar SHA-256 por su resistencia a colisiones y preimagen.

Trabajo futuro

Replicar la cadena en una red de nodos con un algoritmo de consenso (Sistemas Distribuidos), reemplazar FNV-1a por SHA-256, incorporar firmas digitales de los actores y ajustar la dificultad de forma dinámica.

REPARTO DE TAREAS

[Completar: qué parte desarrolló cada integrante.]

REFERENCIAS

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies*. Princeton, NJ, USA: Princeton Univ. Press, 2016.
- [3] A. M. Antonopoulos, *Mastering Bitcoin*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [4] G. Fowler, L. C. Noll, and K.-P. Vo, "FNV hash," [Online]. Available: <http://www.isthe.com/chongo/tech/comp/fnv/>